Relatório Técnico

Projeto – Markdown Simplifcado

Curso: Técnico Especialista em Tecnologias e Programação de Sistemas de Informação
Unidade Curricular: UC00618 - Criar aplicações em linguagem de programação Python

Formando: Hernâni Arriscado
Formador: João Galamba

Índice

1. **Introdução e Objetivos**
   1.1. Contextualização do Projeto 1.2. Estrutura do Relatório 1.3. Objetivos do Projeto

2. **Parte I: Fundamentação e Desenho Arquitetural**
   2.1. Arquitetura Frontend-Backend (Separação de Preocupações)
   2.2. Máquinas de Estados Finitas (FSM) no Parsing de Texto
   2.3. Padrões de Projeto (Design Patterns) Aplicados
   2.4. Processamento Orientado a Linhas e Eficiência Espacial
   2.5. Resolução de Ambiguidades e Precedência no Markdown

3. **Parte II: Implementação Técnica e Engenharia**
   3.1. Interface de Linha de Comando (CLI) com docopt
   3.2. Tratamento Robusto de Exceções e Sinais POSIX
   3.3. Análise Léxica Otimizada via Expressões Regulares (Lexer)
   3.4. Algoritmos Utilitários e Manipulação de Ponteiros de Memória
   3.5. Desenvolvimento das Funcionalidades Avançadas
      3.5.1. Processamento Inline: Cascata Sequencial e Lambdas
      3.5.2. Otimização Computacional: Retenção de Estado e Regex Engine
      3.5.3. Compilação Dinâmica e Tipologia de Listas (Ordered/Unordered)
      3.5.4. Gestão de Estado Segura via Enumerações (Enum)
      3.5.5. Consolidação do Padrão Strategy: Desacoplamento Absoluto

4. **Diagramas e Fluxogramas**
   4.1. Visão Arquitetural (Geral e Classes)
   4.2. Comportamento e Estados (FSM Principal e Listas)
   4.3. Fluxo de Execução (Ciclo Principal e Elementos Inline)
   4.4. Evolução do Projeto (Parte 1 vs Parte 2)

5. **Conclusão**
   5.1. Síntese de Funcionalidades Implementadas
   5.2. Valores Arquiteturais e Superação de Desafios
   5.3. Trabalho Futuro e Objetivos Residuais
   5.4. Considerações Finais

6. **Bibliografia**

1. Introdução e Objetivos

O presente relatório documenta detalhadamente o ciclo de desenvolvimento de um Compilador de Markdown Simplificado para HTML, concebido e implementado de raiz na linguagem Python. Este projeto surge no contexto da unidade curricular 618: "Criar Aplicações em Linguagem de Programação Python", funcionando como o veículo principal para a transposição de conceitos teóricos complexos — inerentes ao processamento de linguagens — para uma solução de software prática, funcional e de elevada robustez.

1.1. Contextualização do Projeto

O Markdown consolidou-se indiscutivelmente como o padrão de facto para a documentação técnica moderna. A sua natureza de "marcação leve" prioriza a legibilidade humana sem sacrificar a expressividade estrutural. Desde repositórios no GitHub a sofisticados Sistemas de Gestão de Conteúdo (CMS), a ubiquidade desta linguagem torna a criação de ferramentas de parsing e conversão um desafio de engenharia de software extremamente relevante e atual.

A solução aqui apresentada foca-se na tradução eficiente e precisa dos elementos sintáticos essenciais do Markdown. O sistema foi desenhado para processar duas categorias fundamentais de formatação:

- **Elementos de Bloco:** Cabeçalhos, parágrafos, blocos de código e listas (ordenadas e não-ordenadas).
- **Elementos em Linha (Inline):** Formatações de ênfase (negrito, itálico), hiperligações e imagens.

O motor de compilação garante que a semântica e a estrutura lógica do documento original sejam rigorosamente preservadas durante a sua transição e renderização final para a linguagem de marcação HTML (HyperText Markup Language).

1.2. Estrutura do Relatório

Para refletir fielmente a progressão metodológica, desde a aprendizagem base até à construção do artefato de software, este documento encontra-se organizado em dois eixos fundamentais:

Parte I - Fundamentação e Análise: Focada no estudo preliminar, na análise de vídeos demonstrativos, no levantamento de requisitos e na compreensão profunda dos fundamentos teóricos que regem o processamento e a compilação de linguagens estruturadas.

Parte II - Implementação e Engenharia: Dedicada exclusivamente ao desenvolvimento prático. Esta secção detalha a lógica de parsing, a gestão de estados de transição e a implementação de algoritmos para lidar com as funcionalidades mais avançadas do compilador, tais como o aninhamento de estruturas e a resolução de ambiguidades de sintaxe.

1.3. Objetivos do Projeto

O desenvolvimento deste compilador não visa apenas a obtenção de um produto final funcional, mas sim a consolidação de competências cruciais em engenharia de software. O projeto foi orientado pelos seguintes objetivos estratégicos:

Arquitetura de Compiladores: Compreender e aplicar os princípios fundamentais da separação de responsabilidades entre o Frontend (responsável pela análise léxica e sintática do texto fonte) e o Backend (responsável pela geração do código destino, o HTML).

Sistemas de Parsing Híbridos: * Implementar Máquinas de Estados Finitas (FSM) para o reconhecimento iterativo e preciso de padrões de texto estruturado (elementos de bloco).

- Aplicar **Expressões Regulares (Regex)** de forma otimizada para a deteção e substituição ágil e segura de elementos em linha, lidando com capturas de grupos e padrões não-gananciosos (non-greedy).

Modularidade e Design de Software: Desenvolver um sistema altamente extensível, recorrendo a princípios da Programação Orientada a Objetos (POO) e Padrões de Projeto (Design Patterns). O objetivo é garantir que a futura adição de novas tags de marcação ou a integração de novos formatos de exportação ocorra sem necessidade de refatorização profunda da estrutura base.

Eficiência Algorítmica e Resiliência: Garantir a robustez no processamento de texto. Isto inclui a prevenção de falhas críticas (como loops infinitos perante formatações mal fechadas ou sintaxe inválida) e a minimização da complexidade temporal e espacial durante o ciclo de conversão.

Documentação e Decisão Arquitetural: Justificar de forma clara todas as decisões de arquitetura tomadas, evidenciando o raciocínio analítico subjacente à escolha de estruturas de dados específicas (como pilhas para controlo de aninhamento) e abordagens algorítmicas no ecossistema Python.

2. Parte I: Fundamentação e Desenho Arquitetural

A análise teórica forneceu os alicerces essenciais para a engenharia deste compilador, orientando de forma pragmática as decisões tomadas antes da escrita da primeira linha de código.

2.1. Arquitetura Frontend-Backend

O paradigma *Frontend-Backend* revelou-se a estratégia mais robusta para a construção do compilador, promovendo o princípio da Separação de Preocupações (*Separation of Concerns*). Nesta topologia, o sistema é estritamente bifurcado: o *Frontend* assume a responsabilidade da leitura e interpretação semântica do Markdown operando de forma contínua (*direct-rendering approach*); o *Backend*, por sua vez, atua como um recetor passivo que traduz essas diretivas para o formato final. Esta segregação introduz vantagens vitais:

- **Elevada Modularidade:** O motor de análise torna-se agnóstico em relação à plataforma de destino.
- **Extensibilidade Horizontal:** Facilita a acoplagem de novos geradores de saída (e.g., LaTeX ou PDFs) sem necessidade de reescrever a complexa lógica de leitura.

2.2. Máquinas de Estados Finitas (FSM)

Para o *parsing* de linguagens de marcação sensíveis ao contexto — onde o significado de um símbolo depende do bloco em que está inserido —, as Máquinas de Estados Finitas provaram ser a ferramenta analítica ideal. Ao definir matematicamente os estados possíveis do documento, o compilador reage a gatilhos textuais (como quebras de linha ou mudanças de indentação) para transitar de forma determinística entre contextos lógicos (ex: estar fora de um bloco vs. estar a processar ativamente uma lista).

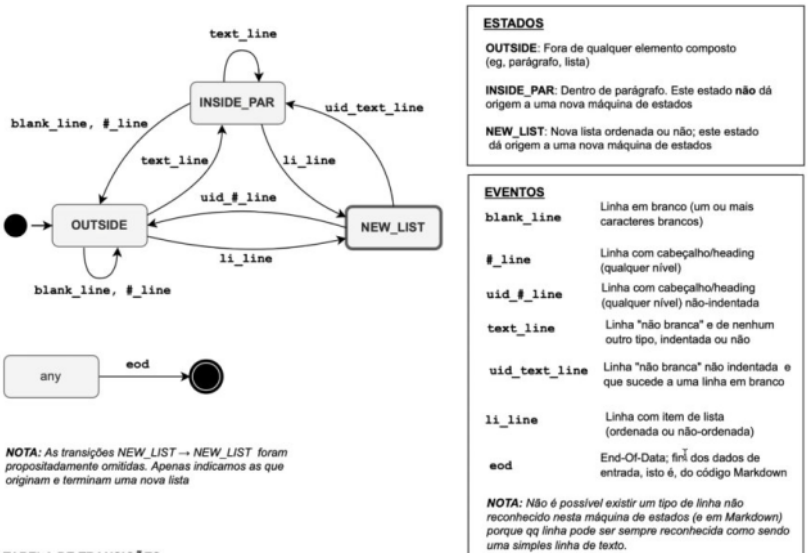


Imagem 1: Exemplo de FSM do Markos.py.

2.3. Padrões de Projeto (Design Patterns)

A adoção de Padrões de Projeto foi imperativa para garantir que a base de código permanecesse coesa e escalável:

- **Padrão Strategy:** Estabelece um contrato rígido através de interfaces abstratas, obrigando qualquer gerador de saída a implementar métodos padronizados, isolando a lógica de conversão.
- **Padrão Visitor / Single Dispatch:** Abordagem polimórfica que permite processar diferentes tipos de nós estruturais eliminando longas cadeias condicionais.
- **Padrão State:** Encapsulamento dos comportamentos da Máquina de Estados, prevenindo a colisão de estados lógicos.

2.4. Processamento Orientado a Linhas e Eficiência Espacial

Em oposição ao carregamento integral do documento para a memória RAM (uma prática ineficiente para ficheiros massivos), o sistema foi desenhado para atuar sobre streams de dados. O compilador consome o texto linha a linha, mantendo a pegada de memória reduzida e constante. Para viabilizar esta arquitetura em estruturas complexas, planeou-se um mecanismo de retrocesso (rewind) que permite devolver o controlo de linhas não processadas à máquina de estados após uma leitura de antecipação (lookahead).

2.5. Resolução de Ambiguidades no Markdown

A conversão de formatações inline (como negritos dentro de hiperligações) exige uma ordem de precedência estrita. Arquiteturalmente, definiu-se que o motor deveria resolver os elementos em cascata (imagens, hiperligações e, por fim, ênfases visuais) utilizando quantificadores não-gananciosos para evitar fusões indevidas de formatações adjacentes, dispensando parsers recursivos pesados.

3. Parte II: Implementação Técnica e Engenharia

Esta secção detalha a transposição da teoria para a linguagem Python, evidenciando o código, as otimizações e a mitigação de falhas construídas durante o ciclo de desenvolvimento.

3.1. Interface de Linha de Comando (CLI) com docopt e Gestão de I/O

A orquestração do programa no terminal é integralmente gerida através da biblioteca docopt, que adota o elegante paradigma de Docstring-Driven Design. Em oposição à configuração imperativa e verbosa típica de bibliotecas como o argparse, a interface deste compilador é gerada dinamicamente a partir da própria documentação (docstring) presente no módulo principal markos.py.

```
def main():
    doc = f"""
    Markos is Markdown converter. It converts from Simplified Markdown
    to HTML.

    Usage:
        {sys.argv[0]} [-s STYLE_SHEET] [-p] [INPUT_FILE] [OUTPUT_FILE]

    Options:
        INPUT_FILE          Markdown source file. Default: stdin
        OUTPUT_FILE         Output source file. Default: stdout
        -h, --help          This help
        -p, --pretty        Prettify HTML output
        -s STYLE_SHEET, --style-sheet=STYLE_SHEET   Use this STYLE_SHEET
    """
    args = docopt(doc)

    style_sheet = args['--style-sheet']
    pretty_print = args['--pretty']
```

Imagem 2 - Definição da CLI e extração de argumentos via biblioteca docopt.

As principais especificidades arquiteturais da CLI incluem:

- **Agnosticismo de Entrada/Saída (I/O) com Suporte UTF-8:** Suporte nativo para leitura e escrita em ficheiros físicos, munido de um mecanismo de fallback inteligente (gerido no módulo utils.py) para os standard streams do sistema (stdin e stdout). A implementação força a codificação utf-8 explícita nas operações de I/O, neutralizando discrepâncias de acentuação e encoding nativas de sistemas operativos como o Windows. Esta flexibilidade permite que o compilador seja embutido em pipelines de terminal, suportando redirecionamentos clássicos do Unix (como < e >).

- **Injeção de Estilo (-s / --style-sheet):** Capacidade de referenciar folhas de estilo CSS externas no documento final. O sistema injeta dinamicamente a tag <link> apropriada, garantindo que o HTML gerado já possua a camada de apresentação visual acoplada.

- **Renderização Formatada (-p / --pretty):** Opção de formatação que delega o pós-processamento da string final para o HTMLFormatter da biblioteca BeautifulSoup. Esta integração reestrutura o HTML de saída com a correta indentação e quebras de linha (human-readable), uma funcionalidade crucial para a depuração visual e validação estrutural do código gerado.

3.2. Tratamento Robusto de Exceções e Sinais POSIX

A resiliência e a previsibilidade do sistema foram tratadas como prioridades. O invólucro de execução foi protegido por blocos try-except desenhados para capturar anomalias sem expor stack traces agressivos ao utilizador final, canalizando semanticamente os erros para a saída de erro padrão (sys.stderr).

```
try:
    in_file = from_file_or_stdin(args['INPUT_FILE'])
    out_file = to_file_or_stdout(args['OUTPUT_FILE'])

    backend = HTMLBackend(out_file, style_sheet, pretty_print)

    with in_file, out_file, closing(backend):
        mkd_compiler = MarkdownCompiler(backend)
        mkd_compiler.compile(in_file)

except FileNotFoundError as ex:
    print(f"File not found: {ex.filename}", file = sys.stderr)
    sys.exit(2)

except PermissionError as ex:
    print(f"Invalid permissions to access file: {ex.filename}", file=sys.stderr)
    sys.exit(13)

except Exception as ex:
    print(f"An error has ocurred:\n{ex.args}\n\n", file=sys.stderr)
    raise ex
```

Imagem 3 - Gestão de Exceções I/O e mapeamento de sinais POSIX.

O ciclo de vida da aplicação mapeia os seus estados de terminação em conformidade com as convenções POSIX (Exit Codes):

- **0 (Sucesso Absoluto):** O documento foi lido, processado pela máquina de estados e exportado sem qualquer anomalia.
- **2 (ENOENT - No such file or directory):** O sistema interceta a ausência do ficheiro de entrada (FileNotFoundError) logo no arranque, abortando a execução antes de iniciar qualquer alocação desnecessária de memória.
- **13 (EACCES - Permission denied):** Tratamento proativo de falhas de permissão de I/O (PermissionError), acionado caso o sistema operativo negue privilégios de leitura do ficheiro fonte ou de escrita no diretório de destino.

3.3. Análise Léxica Otimizada via Expressões Regulares

O motor inicial de parsing (o Lexer do compilador) foi construído em torno de Expressões Regulares (Regex) estrategicamente desenhadas para atuar como sensores de alta velocidade.

Estes padrões alimentam a Máquina de Estados, permitindo:

- Detetar linhas vazias ou compostas apenas por espaços em branco (^[\t\n\r]*), cruciais para sinalizar o encerramento semântico de parágrafos ou o fim de blocos de listas.
- Identificar dinamicamente os diferentes níveis de cabeçalhos (#), tolerando variações de indentação estrutural no início da string.
- Reconhecer o marcador específico de título do documento (delimitado pelo caráter !).

```
class MarkdownCompiler:
    INLINE_TITLE_MARKER = '!'
    INLINE_HEADING_MARKER = '#'

    BLANK_LINE = re.compile('^[ \t\n\r]*')
    UNINDENT_HEADING_LINE = re.compile(fr'\s?{INLINE_HEADING_MARKER}{"{1,6}"}(\s+.*)?')
    INDENT_HEADING_LINE = re.compile(fr'\s{"{2,}"}{INLINE_HEADING_MARKER}{"{1,6}"}(\s+.*)?')
    UNINDENT_TEXT_LINE = re.compile(r'\s?\S.*')
    INDENT_TEXT_LINE = re.compile(r'\s{2,}\S.*')
    TITLE_LINE = re.compile(fr'{INLINE_TITLE_MARKER}.*\S.*{INLINE_TITLE_MARKER}')

    LIST_ITEM_LINE = re.compile(r'\s{0,3}(?:[-*]|\d+[.)])(\s+.*)?')
```

Imagem 4 - Declaração de constantes e compilação prévia das Expressões Regulares estruturais no escopo da classe MarkdownCompiler.

3.4. Algoritmos Utilitários e Manipulação Direta de Memória

O módulo utils.py demonstra um domínio algorítmico avançado sobre a manipulação de fluxos de dados, contornando a necessidade de carregar ficheiros inteiros para a memória. Como o compilador consome os dados através de uma *stream* contínua (um descritor de ficheiro ou um objeto StringIO), o motor de *parsing* precisa frequentemente de "espreitar" a linha seguinte (*lookahead*).

Para tal, desenvolveram-se algoritmos utilitários vitais:

- **Manipulação Cirúrgica de Ponteiros (rewind_one_line):** Em vez de criar *buffers* temporários, a função calcula o comprimento da linha processada, adiciona um caráter para compensar a quebra de linha (\n) que foi cortada (len(line) - 1), e invoca in_.tell() e in_.seek() para recuar a agulha de leitura matematicamente para o ponto anterior.

- **Contagem Linear Limpa (count_consec):** Algoritmos iterativos customizados que garantem a extração da cardinalidade dos cabeçalhos sem recorrer a bibliotecas externas pesadas.

```
def prettify_html(html_code: str | TextIO, indent = 2) -> str:
    soup = BeautifulSoup(html_code, features = 'html.parser')
    return soup.prettify(formatter = HTMLFormatter(indent = indent))

def rewind_one_line(in_: TextIO, line: str):
    in_.seek(in_.tell() - len(line) - 1, 0)

def from_file_or_stdin(file_path: str | None) -> TextIO:
    if file_path:
        with open(file_path, 'rt', encoding='utf-8') as f:
            return StringIO(f.read())
    return sys.stdin
```

Imagem 5 - Manipulação direta do ponteiro de memória na stream de leitura.

3.5. Parte III: Desenvolvimento das Funcionalidades Avançadas

A segunda fase de implementação elevou a complexidade do projeto, dotando o compilador de mecanismos robustos para lidar com a natureza imprevisível das listas mistas e garantindo a injeção estritamente segura de elementos em linha (*inline*).

3.5.1. Processamento Inline com Delegação Lambda e Cascata Sequencial

A conversão de elementos *inline* exigiu uma abordagem arquitetural de alta segurança, dado que implicam substituições intra-textuais simultâneas e frequentemente aninhadas. A rotina _parse_inline() implementa a seguinte arquitetura:

- **Cascata Sequencial Estrita:** Previne colisões processando primeiro as imagens (!\[...), seguidas de hiperligações, negritos e, por fim, itálicos.
- **Delegação via Funções Anónimas (*Lambda*):** A extração de dados é realizada através de grupos de captura Regex (m.group(1), m.group(2)). Em vez de o Frontend concatenar *strings* HTML manualmente, a função re.sub recebe funções *lambda* que passam os grupos imediatamente como argumentos para os métodos do Backend (como format_link).
- **Avaliação em Curto-Circuito (*Short-circuit Evaluation*):** Para atributos opcionais (como o título de um link), utilizam-se construções defensivas como m.group(3) or '', neutralizando o risco de injetar objetos None na árvore do documento final.

```python
def _parse_inline(self, line: str) -> str:
    line = re.sub(
        r'!\[([^\]]*)\]\(([^)\s]+)(?:\s+"([^"]+)")?\)',
        lambda m: self._backend.format_image(m.group(1), m.group(2), m.group(3) or ''),
        line
    )
    line = re.sub(
        r'(?<!!)\[([^\]]*)\]\(([^)\s]*)(?:\s+"([^"]+)")?\)',
        lambda m: self._backend.format_link(m.group(1), m.group(2), m.group(3) or ''),
        line
    )
    line = re.sub(
        r'(\*\*|__)(.+?)\1',
        lambda m: self._backend.format_bold(m.group(2)),
        line
    )
    line = re.sub(
        r'(\*|_)(.+?)\1',
        lambda m: self._backend.format_italic(m.group(2)),
        line
    )
    return line
```

Imagem 6 - Processamento sequencial de elementos inline utilizando Regex, funções lambda e avaliação em curto-circuito.

3.5.2. Otimização Computacional e Retenção de Estado (Regex Engine)

Em ficheiros Markdown de grande magnitude, a instanciação repetitiva do motor de Expressões Regulares dentro de ciclos de leitura intensivos pode degradar severamente a *performance*. Para mitigar essa latência:

- **Compilação Estática:** Padrões complexos, como LIST_ITEM_LINE e INDENT_HEADING_LINE, são submetidos a re.compile() no escopo estático da classe MarkdownCompiler, alocando a sua memória uma única vez.

- **Reaproveitamento Dinâmico:** Estes objetos são reaproveitados em todo o ciclo de vida do programa (através de invocações como matches()), poupando milhares de ciclos de CPU na reavaliação de padrões.

```
class MarkdownCompiler:
    [...]
    INLINE_HEADING_MARKER = '#'

    BLANK_LINE = re.compile('^[ \t\n\r]*')
    UNINDENT_HEADING_LINE = re.compile(fr'\s?{INLINE_HEADING_MARKER}{"{1,6}"}(\s
    INDENT_HEADING_LINE = re.compile(fr'\s{"{2,}"}{INLINE_HEADING_MARKER}{"{1,6}'
    LIST_ITEM_LINE = re.compile(r'\s{0,3}(?:[-*]|\d+[.)])(\s+.*)?')
    [...]

    def _is_heading_line(self, line: str) -> bool:
        return (
                matches(self.UNINDENT_HEADING_LINE, line) or
                matches(self.INDENT_HEADING_LINE, line)
        )
```

Imagem 7 - Otimização computacional: compilação estática de Expressões Regulares no escopo da classe e o seu reaproveitamento dinâmico através da função matches.

3.5.3. Compilação Dinâmica e Tipologia de Listas

A necessidade de suportar listas ordenadas, não ordenadas e transições mistas originou um refinamento estrutural profundo na sub-Máquina de Estados de listas. A sua lógica atua em três eixos:

- **Extração Tipológica em Tempo Real:** O sistema extrai o prefixo do item (utilizando a Regex ^\s{0,3}\d+[.)]) e regista o estado na propriedade booleana is_ordered do objeto abstrato MarkdownList.

- **Contenção de Desvios Estruturais:** A lógica confronta a tipologia da linha atual com a da linha subsequente. Perante uma transição abrupta (e.g., de 1. para -), a máquina quebra o ciclo, encerra o bloco semântico e devolve o controlo ao estado superior (NEW_LIST).

- **Renderização Semântica Isolada:** Este mecanismo assegura que não ocorrem sobreposições nem corrupção na renderização final de <ol> e <ul>.

```python
def _compile_list(self, in_: TextIO):
    line = in_.readline()[:-1]
    [...]
    mkd_list = MarkdownList()
    is_current_list_ordered = bool(re.match(r'^\s{0,3}\d+[.)]', line))
    mkd_list.is_ordered = is_current_list_ordered
    [...]
    while line := in_.readline():
        line = line[:-1]
        is_next_item_ordered = bool(re.match(r'^\s{0,3}\d+[.)]', line))
        is_list_item_match = matches(self.LIST_ITEM_LINE, line)
        [...]
        elif state is ListState.LIST_ITEM and is_list_item_match:
            if is_next_item_ordered != is_current_list_ordered:
                rewind_one_line(in_, line)
                break
```

Imagem 8 - Extração em tempo real da tipologia da lista e contenção de desvios estruturais na sub-máquina de estados *_compile_list.*

3.5.4. Gestão de Estado Tipada e Segura (Enumerações)

Para erradicar a fragilidade introduzida pela utilização de *Magic Strings* (e.g., verificar estados com if state == "INSIDE_PAR"), a orquestração do compilador foi blindada logicamente:

- **Erradicação de Fragilidades Lógicas:** Utilização do módulo nativo enum do Python para criar enumeradores estritos como CompilerState e ListState.
- **Conceito *Fail-Fast*:** Qualquer tentativa de transitar o compilador para um estado não previsto resulta num erro de sintaxe imediato durante o desenvolvimento, assegurando um fluxo determinístico e totalmente imune a erros de digitação (*typos*) na base de código.

```python
def compile(self, in_: TextIO | str):
    if isinstance(in_, str):
        in_ = StringIO(in_)

    backend = self._backend
    title = self._read_title(in_)
    backend.open_document(title)

    CompilerState = Enum('CompilerState', 'OUTSIDE INSIDE_PAR NEW_LIST')
    state = CompilerState.OUTSIDE

    while line := in_.readline():
        line = line[:-1]

        if state is CompilerState.OUTSIDE and self._is_heading_line(line):
            self._new_heading(line)
```

Imagem 9 - Instanciação da Máquina de Estados principal com base no módulo nativo Enum.

3.5.5. Desacoplamento Absoluto via Padrão Strategy

O clímax desta engenharia reside na consolidação do Padrão Strategy. A sua aplicação garante a concretização da premissa base do projeto:

- **Abstração Hermética do Destino:** O objeto MarkdownCompiler (o Frontend) não contém, em momento algum, conhecimento sobre a sintaxe do formato de destino.
- **Comunicação Polimórfica:** A transferência de dados ocorre através de invocações estritas à interface abstrata MarkdownBackend, utilizando métodos como backend.open_list() ou backend.format_bold().
- **Extensibilidade *Plug-and-Play*:** Substituir a saída HTML por JSON de uma AST, documentação LaTeX ou XML exige unicamente a injeção de uma nova classe derivada, mantendo o imenso e complexo motor de *parsing* completamente inalterado.

```python
class MarkdownBackend(ABC):
    @abstractmethod
    def open_document(self, title = ''):
        pass

    @abstractmethod
    def close_document(self):
        pass

    @abstractmethod
    def open_heading(self, level: int):
        pass

    @abstractmethod
    def close_heading(self, level: int):
        pass

    @abstractmethod
    def new_text_line(self, line: str):
        pass
```

```python
    @abstractmethod
    def close_list(self):
        pass

    @abstractmethod
    def format_bold(self, text: str):
        pass

    @abstractmethod
    def format_italic(self, text: str):
        pass

    @abstractmethod
    def format_code(self, text: str):
        pass

    @abstractmethod
    def format_link(self, text: str, url: str):
        pass
```

Imagens 9 e 10 - Definição da interface abstrata MarkdownBackend, estabelecendo o contrato obrigatório para qualquer gerador de saída.

```python
class MarkdownCompiler:
    [...]
    def __init__(self, backend: MarkdownBackend):
        self._backend = backend

    [...]

    def _new_heading(self, line_with_markers: str):
        backend = self._backend
        text, level = self._parse_heading(line_with_markers)

        backend.open_heading(level)
        backend.new_text_line(self._parse_inline(text))
        backend.close_heading(level)
```

Imagem 11 - Aplicação do Padrão Strategy: injeção de dependência no construtor e delegação polimórfica no método _new_heading do Frontend.

4. Diagramas e Fluxogramas

4.1. Visão Arquitetural

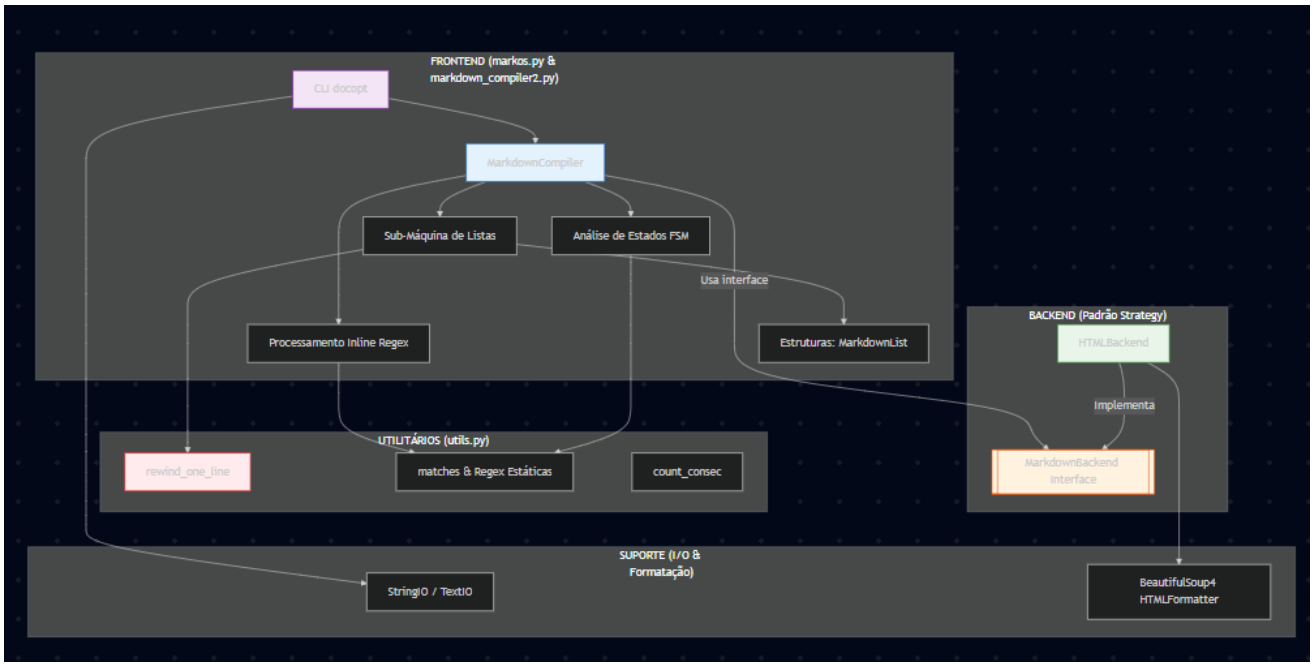


Diagrama 1 - **Diagrama de Arquitetura Geral** (Frontend vs Backend e Utilitários).

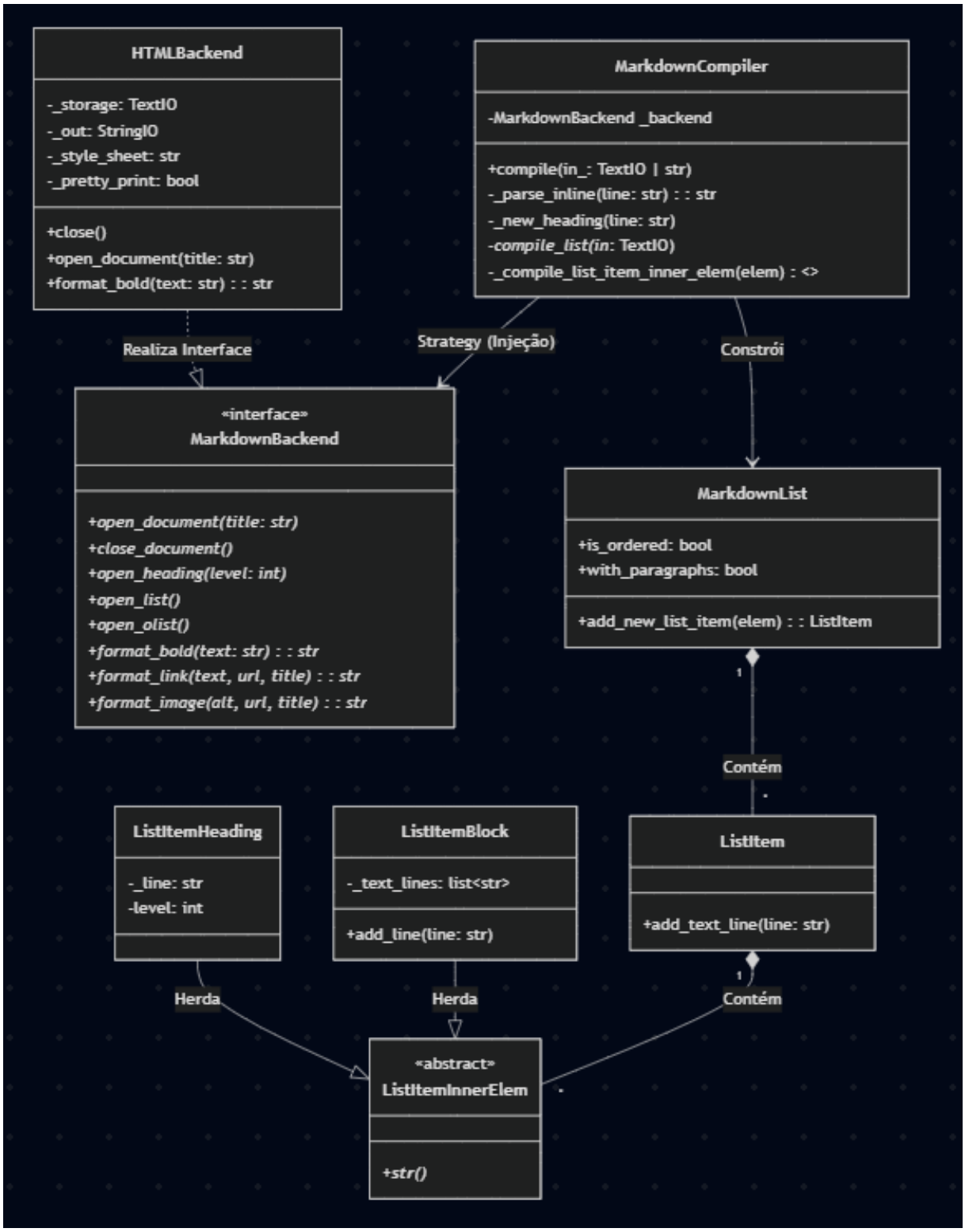


Diagrama 2 - **Diagrama de Classes Principais** (Relações, heranças e interfaces do Padrão Strategy).

4.2. Comportamento e Estados (FSM)

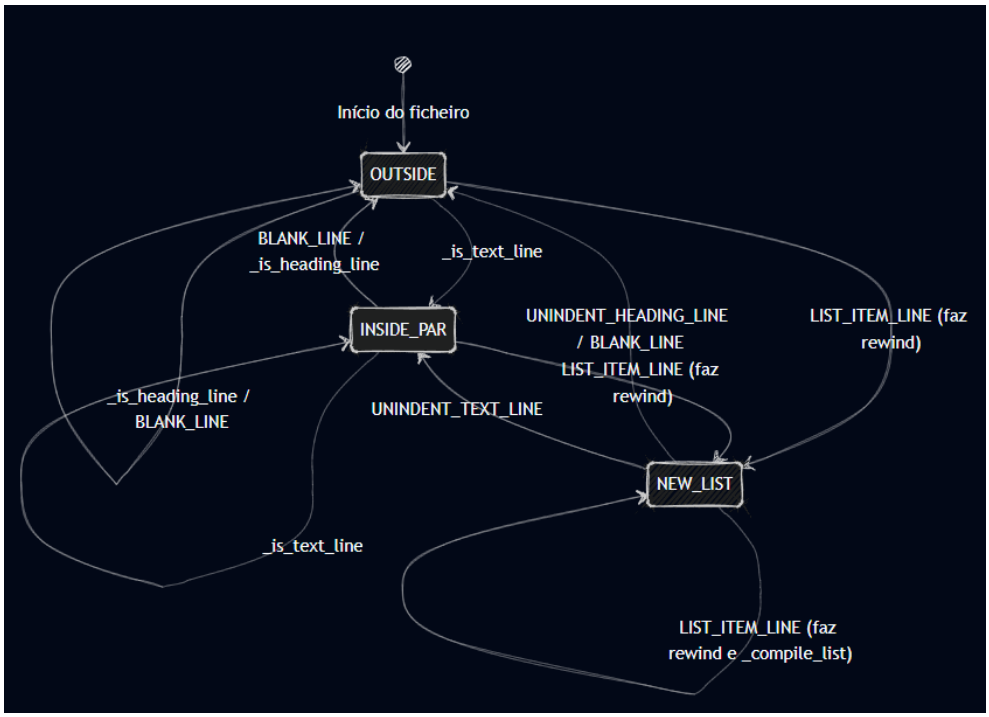


Diagrama 3 - **Diagrama da Máquina de Estados Principal** (Transições entre OUTSIDE, INSIDE_PAR, NEW_LIST).

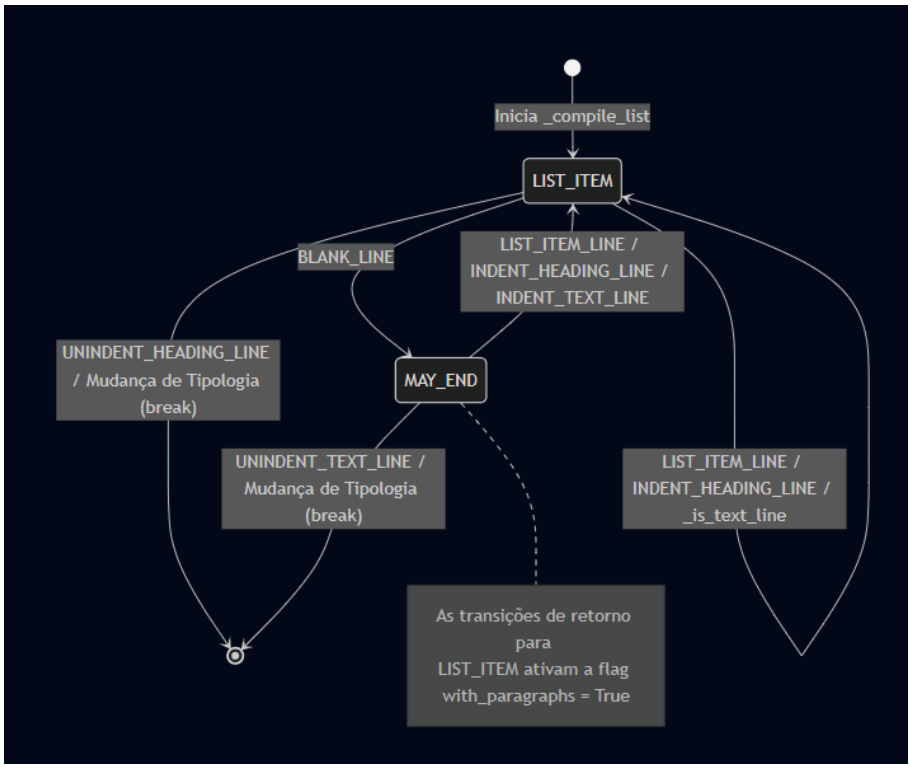
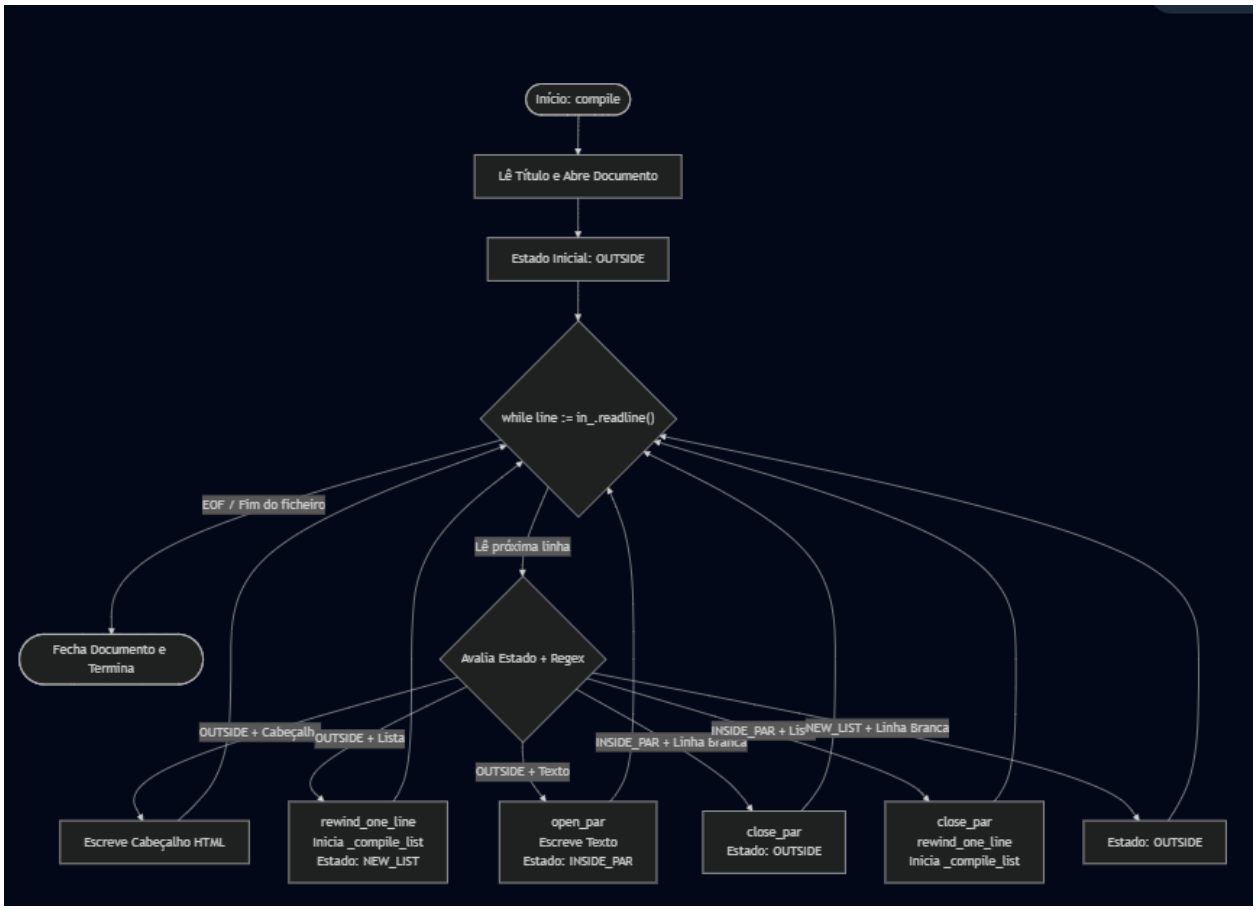


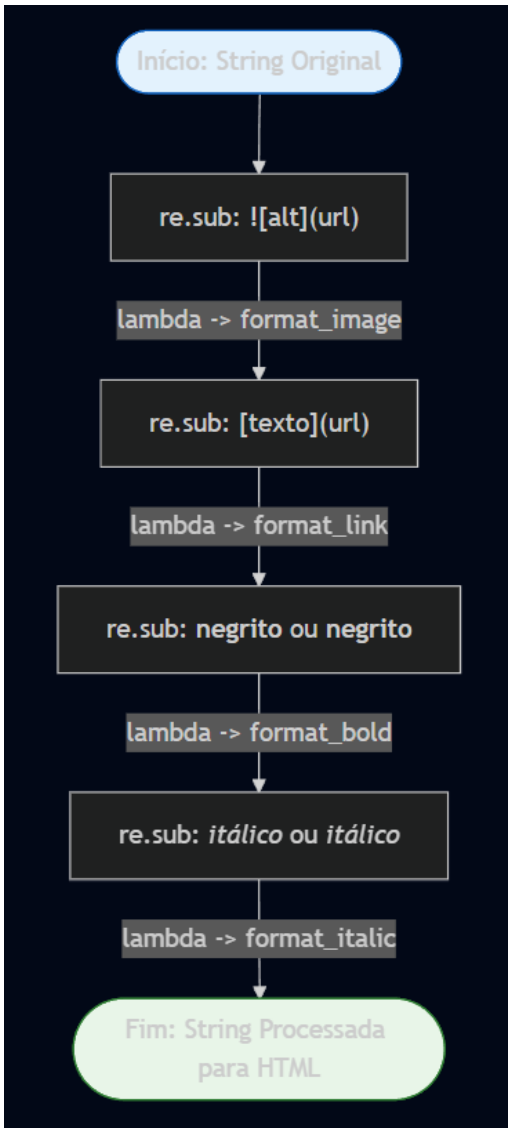
Diagrama 4 - **Diagrama da Sub-Máquina de Listas (Transições entre LIST_ITEM e MAY_END).**

4.3. Fluxo de Execução



Fluxograma 1 - **Fluxograma Principal** (O ciclo while de leitura linha a linha e delegação).

Fluxograma 2 - **Fluxograma de Processamento de Listas** (A extração de tipologia e contenção de desvios).



Fluxograma 3 - **Fluxograma de Elementos Inline (A cascata sequencial de Expressões Regulares).**

5. Conclusão

O ciclo de desenvolvimento documentado neste relatório consubstancia uma jornada enriquecedora, tanto do ponto de vista da engenharia de software como da consolidação pedagógica. A transposição dos conceitos teóricos de processamento de linguagens para a linguagem Python permitiu materializar o conhecimento abstrato em algoritmos palpáveis, consolidando o domínio sobre a Programação Orientada a Objetos (POO), padrões de projeto e otimização de fluxos de memória.

A escolha por uma arquitetura de **Renderização Direta** (*Direct Rendering*), em detrimento da construção de uma Árvore de Sintaxe Abstrata (AST) global, provou ser uma estratégia eficaz para as restrições do projeto. O sistema demonstrou uma *performance* robusta, uma pegada de memória otimizada e uma modularidade que garante a sua evolução futura.

5.1. Síntese de Funcionalidades Implementadas

O projeto culminou na entrega de um compilador plenamente funcional e de elevada robustez, suportando o conjunto de requisitos exigidos nas Partes I e II:

- **Motor de Parsing e Máquinas de Estado:** Implementação de FSMs tipadas (via módulo enum) para o processamento resiliente de blocos estruturais, incluindo cabeçalhos (níveis 1 a 6 detetados dinamicamente via Regex) e parágrafos com formatação semântica estrita.

- **Processamento Complexo de Listas:** Suporte integral a listas não-ordenadas (marcadores - e *) e listas ordenadas (marcadores 1. e 1)), geridas por uma sub-máquina de estados capaz de resolver desvios de tipologia e injetar parágrafos internos (*loose lists*).

- **Resolução Analítica de Elementos Inline:** Conversão segura de negrito, itálico, hiperligações e imagens recorrendo a grupos de captura em Regex, garantindo a validação de atributos opcionais e a delegação da formatação via funções *lambda*.

- **Interface de Utilizador e I/O:** CLI sofisticada alimentada por docopt, oferecendo processamento de *streams* agnóstico (stdin/stdout), integração de folhas de estilo externas (CSS) e pós-processamento para estruturação visual do código de saída (HTML pretty-print).

5.2. Valores Arquiteturais e Superação de Desafios

Do ponto de vista da engenharia, o compilador destacou-se pela aplicação rigorosa de boas práticas:

- **Desacoplamento via Padrão Strategy:** A segregação total entre o *Frontend* analítico e o *Backend* gerador revelou-se o maior trunfo do projeto, permitindo coordenar o fluxo do programa sem dependências diretas com o código HTML final.

- **Controlo de Fluxo e Memória:** A superação da complexidade das estruturas aninhadas foi garantida pela manipulação inteligente de ponteiros de leitura (função rewind_one_line()), evitando o carregamento massivo de dados para a RAM.

- **Segurança e Resiliência:** O sistema foi desenhado para prevenir vulnerabilidades (como ciclos infinitos no parsing) e gerir exceções de I/O de forma proativa, traduzindo-as em códigos de erro padronizados.

5.3. Trabalho Futuro e Objetivos Residuais

Embora o artefacto cumpra todos os requisitos nucleares e supere as expectativas de robustez, alguns vetores de evolução ficaram fora do âmbito desta versão:

- **Funcionalidades Extra:** Não foi implementado o *Backend* alternativo para exportação em LaTeX ou formato JSON (AST).

- **Elementos Markdown Avançados:** O suporte a estruturas como listas aninhadas (*nested lists*), blocos de citação (>), blocos de código e tabelas foi deliberadamente excluído para manter o foco na estabilidade do motor base.

5.4. Considerações Finais

Em suma, o projeto transcende a simples criação de um tradutor de texto. Representa a prova prática de como os dogmas da ciência da computação — planeamento arquitetural, aplicação de *Design Patterns* e mitigação de falhas — são imperativos para a resolução de problemas reais. A atenção meticulosa aos detalhes resultou num sistema não apenas funcional, mas altamente performante, seguro e pronto a escalar.

6. Bibliografia

Para a fundamentação técnica e teórica deste projeto, foram consultadas as seguintes fontes de referência:

Documentação e Especificações Oficiais

- **CommonMark Specification.** (2024). *Standard Markdown Specification*. Disponível em: https://spec.commonmark.org/
- **Python Software Foundation.** (2024). *The Python Standard Library (re, enum, functools, typing)*. Disponível em: https://docs.python.org/3/library/
- **Docopt Organization.** (2024). *Docopt: Command-line interface description language*. Disponível em: http://docopt.org/
- **Beautiful Soup Documentation.** (2024). *Beautiful Soup 4.12: Library for pulling data out of HTML*. Disponível em: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Engenharia de Software e Padrões de Projeto

- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Referência para os padrões **Strategy**, **State** e **Visitor**).
- **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. (Referência para a separação de responsabilidades e organização de lógica).
- **Ramalho, L.** (2022). *Fluent Python: Clear, Concise, and Effective Programming*. O'Reilly Media. (Referência para a utilização de **@singledispatch** e manipulação eficiente de strings/streams).

Processamento de Linguagens e Compiladores

- **Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D.** (2006). *Compilers: Principles, Techniques, and Tools* (Dragon Book). Pearson. (Referência para os conceitos de **Análise Léxica** e **Máquinas de Estados Finitas**).
- **Friedl, J. E. F.** (2006). *Mastering Regular Expressions*. O'Reilly Media. (Referência para a otimização de motores de Regex e prevenção de retrocessos catastróficos).

Metodologia Pedagógica

- **Galamba, J.** (2024-2026). *Material Pedagógico e Gravações da UC00618: Criar Aplicações em Python*. Citeforma. (Base estrutural e lógica da Parte 1 do projeto).